

Atelier IoT

Réf. 000-MonPremierProgramme_v08-2026-001

Description de l'Atelier

Ce projet incontournable permet de programmer votre module IoT en faisant clignoter une LED sur une fréquence donnée et par conséquent valider tout son environnement de développement.

A travers ce projet, l'apprenant va pouvoir :

- Découvrir la carte de prototypage ESP8266 Lolin NodeMCU v3
- Se familiariser avec l'environnement de développement « CLion »
- Appréhender les notions fondamentales de la programmation IoT
- Coder, Builder et Téléverser le programme dans l'ESP8266.
- Contempler le fonctionnement 😊

Matériel nécessaire

1 carte de prototypage ESP8266 Lolin NodeMcu v3.0	
1 câble Micro-USB B/USB A (liaison entre l'ordinateur et l'ESP8266)	

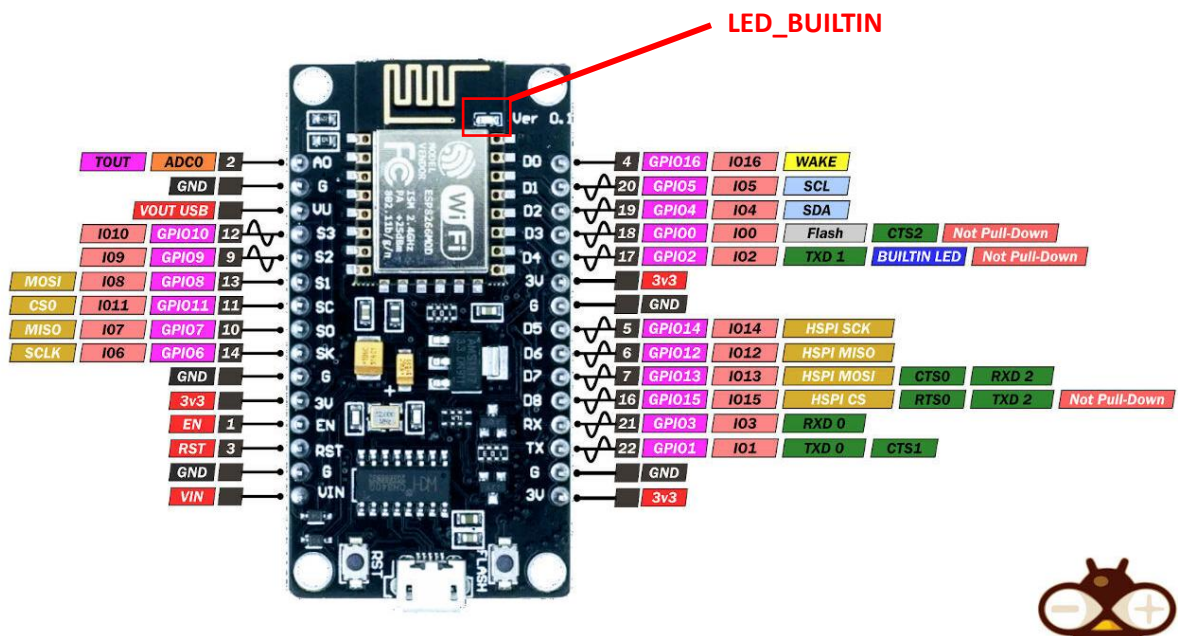
Les Etapes de l'Atelier

Etape 1 – Câblage

Pour ce projet, aucun câblage n'est requis. Nous utiliserons la LED native de notre carte de prototypage.



ESP8266 NodeMcu Lolin v3 / CH340 (brochage)

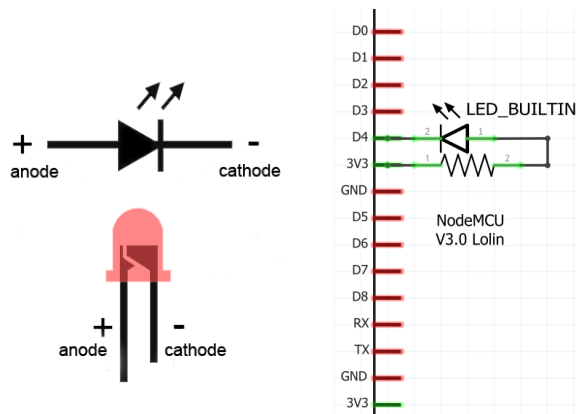


A noter sur la LED native de l'ESP8266

Pour allumer ou éteindre une LED il faut utiliser les deux états possibles :

- HIGH ($\rightarrow V_{cc} = 3.3v$)
- LOW ($\rightarrow Gnd = 0v$)

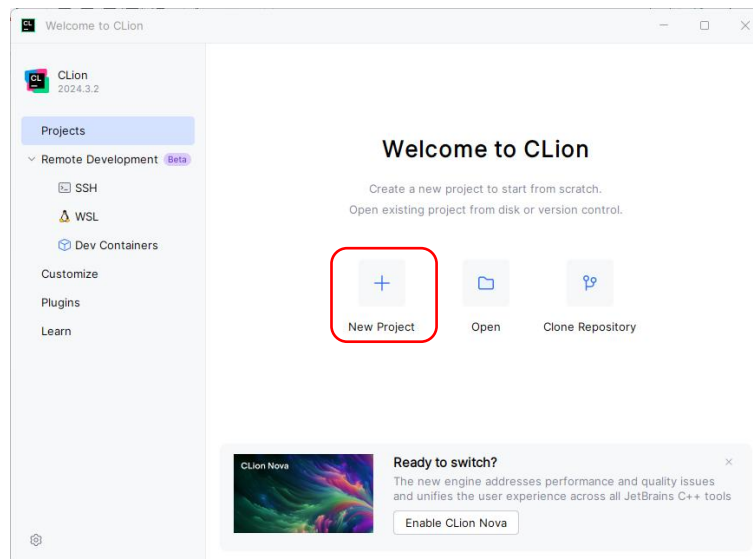
Attention, la LED native qui équipe notre ESP8266, aussi appelée **LED_BUILTIN**, est câblée avec la cathode connectée à la broche D4 (GPIO 2) et l'anode à V_{cc} . Cette LED s'allume donc si l'on passe D4 à LOW et s'éteint si l'on passe D4 à HIGH.



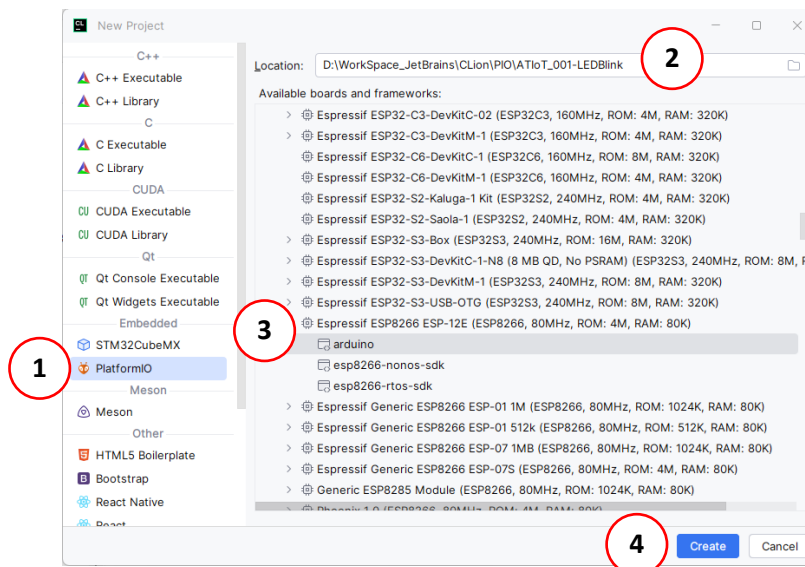
- HIGH ($\rightarrow V_{cc} = 3.3v$) = LED_BUILTIN éteinte
- LOW ($\rightarrow Gnd = 0v$) = LED_BUILTIN allumée.

Etape 2 – Création du projet sur CLion

- Lancez l'IDE CLion et cliquez sur [New Project]

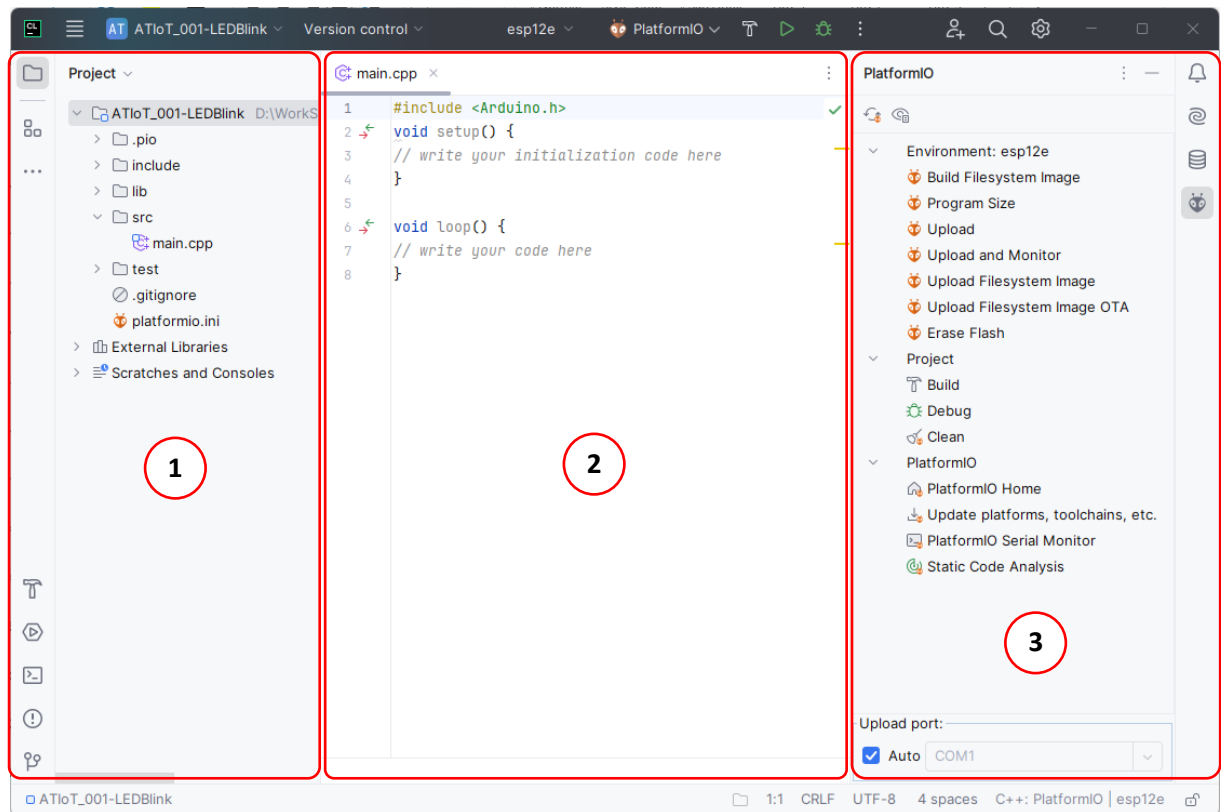


- Dans la fenêtre qui s'affiche :



- 1- Cliquez sur [PlatformIO]
- 2- Saisir le nom du projet, ici « ATIoT_001-LEDBlink »
- 3- Dans la liste des « cartes et frameworks » :
 - a. Cherchez « Espressif »
 - b. Sélectionnez la carte «Espressif **ESP8266 ESP-12E...** »
 - c. Sélectionnez le framework « arduino »
- 4- Cliquez sur [Create]

- Après quelques instants, l'interface de développement apparaît :



- Cette interface est composée de 3 zones principales :

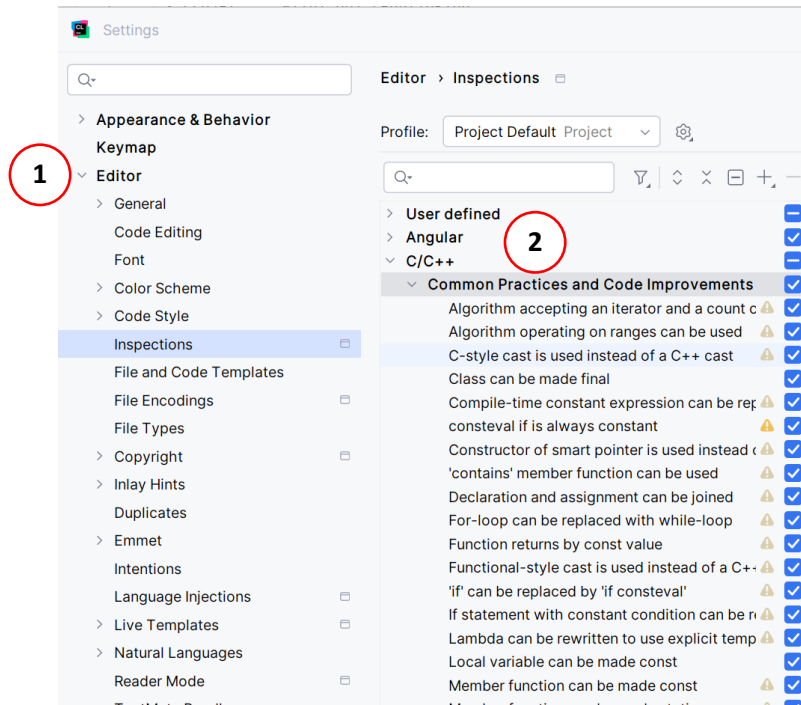
- La zone « Projet » contenant l'arborescence de ce dernier, accessible par 
- La zone « Editeur » contenant plusieurs onglets correspondant chacun à un fichier du projet en cours de visualisation/modification
- La zone « PlatformIO » contenant les diverses fonctions spécifiques à la manipulation de notre IoT, accessible par 

Dans la zone 2, nous découvrons le fichier principal de notre programme nommé « **main.cpp** ». C'est dans ce fichier que nous allons écrire notre code. Il contient déjà les deux fonctions primaires et obligatoires du langage Arduino :

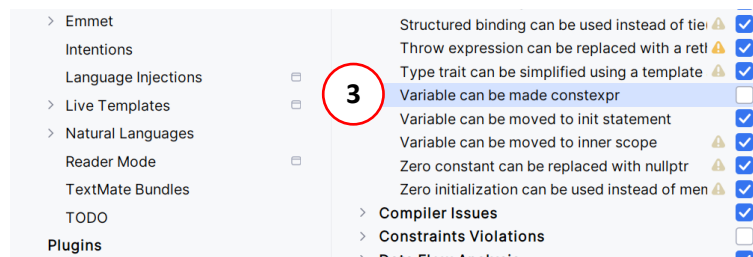
- **setup()**
- **loop()**

Afin de limiter les Warnings de notre IDE, il est nécessaire de désactiver certaines vérifications de ce dernier. Pour cela

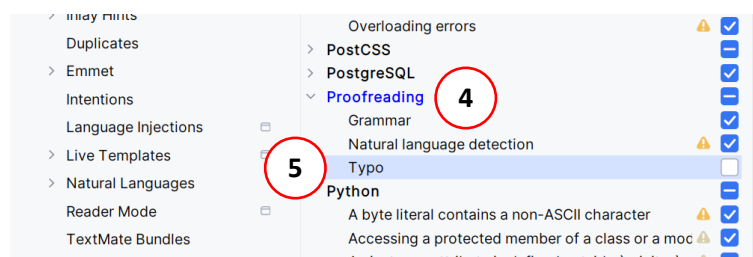
- Appuyez simultanément sur **Ctrl** **Alt** **S** pour afficher la fenêtre des 'Settings'
- Dans cette fenêtre :



1. Sélectionnez « Editor ».« Inspections »
2. Dans la zone « C/C++ », sélectionnez « Common Practices and Code Improvements »
3. Cherchez (en fin de liste) et décochez l'option « Variable can be made constexpr »



4. Dans la zone « Proofreading » (au même niveau que C/C++ mais bien plus bas)
5. Décochez l'option « Typo »



6. Cliquez sur [OK] pour sauvegarder et fermer les « Settings ».

Etape 3 – Etude préliminaire au Codage

L'objectif du programme est simple. La LED native de notre ESP8266 doit s'allumer puis s'éteindre alternativement toutes les 500ms.

Pour cela, place à l'Algorithme de notre programme :

Algorithme : LED_Blinking

Constantes

ALLUMER	booléen :	1
ETEINDRE	booléen :	0

Variables

« duree »	entier
« brocheled »	entier

Début_algo

Affecter a « duree » : 500
Configurer « brocheled » en : sortie

Tant que vrai

 Ecrire sur « brocheled » : ALLUMER
 Attendre « duree »
 Ecrire sur « brocheled » : ETEINDRE
 Attendre « duree »

Fin tant que

Fin_algo

Etape 4 – Premier code

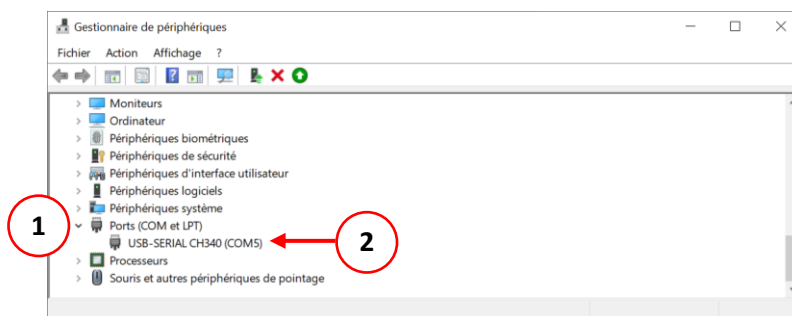
Codez l'algorithme précédent dans votre projet. Cela devrait donner le programme suivant :

*Attention, ici les constantes **ALLUMER** et **ETEINDRE** doivent prendre la valeur correspondante à la réalité liée à l'ESP8266 et sa LED inversée !*

```
main.cpp x
1  #include <Arduino.h>
2
3  int duree;
4  int brocheLed = LED_BUILTIN;
5
6  const bool ETEINDRE = HIGH;
7  const bool ALLUMER = LOW;
8
9  void setup() {
10     duree = 500;
11     pinMode(brocheLed, mode: OUTPUT);
12 }
13 void loop() {
14     digitalWrite( pin: brocheLed, val: ALLUMER);
15     delay(duree);
16     digitalWrite( pin: brocheLed, val: ETEINDRE);
17     delay(duree);
18 }
19
```

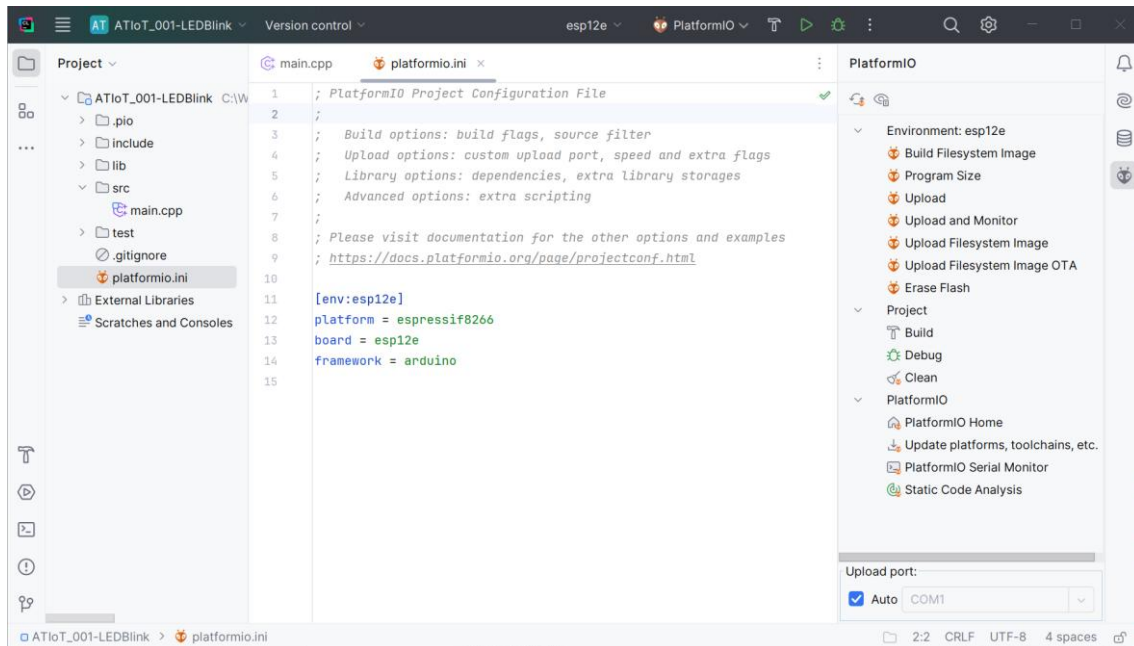
Etape 5 – Connectez la carte, Compilez, Téléversez, Contemplez

- Ouvrez le « gestionnaire de périphériques » de votre ordinateur
- Connectez votre câble Micro-USB B / USB A à votre ESP8266 puis à votre ordinateur
- Dans votre « gestionnaire de périphériques » :



- Ouvrez la zone « Ports (COM et LPT) »
- Cherchez un port dans lequel figure le libellé « CH340 » et relevez son N° (ici '5')

- Dans votre IDE « CLion », ouvrez le fichier « platformio.ini » dont le contenu initial doit être le suivant :



- Ajouter les informations suivantes en remplaçant le N° de COM par celui que vous avez relevé précédemment :

```

1 ; PlatformIO Project Configuration File
2 ;
3 ; Build options: build flags, source filter
4 ; Upload options: custom upload port, speed and extra flags
5 ; Library options: dependencies, extra library storages
6 ; Advanced options: extra scripting
7 ;
8 ; Please visit documentation for the other options and examples
9 ; https://docs.platformio.org/page/projectconf.html
10
11 [env:esp12e]
12 platform = espressif8266
13 board = esp12e
14 framework = arduino
15
16 monitor_port = COM[5]
17 monitor_speed = 115200
18
19 upload_port = COM[5]
20 upload_speed = 115200

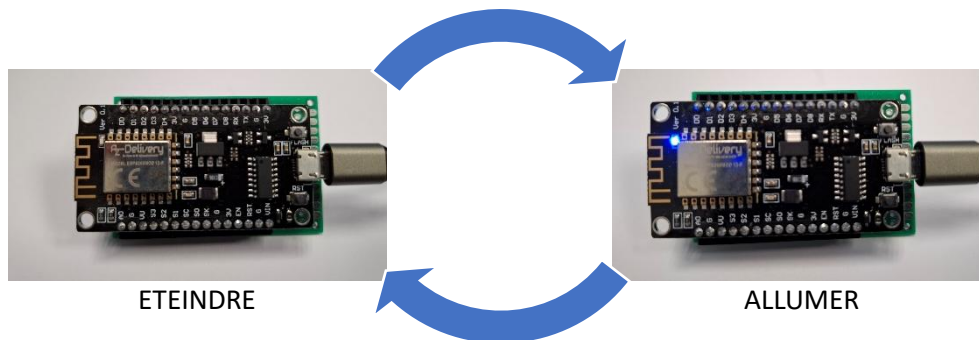
```

- Maintenant :

- Compilez votre code en cliquant sur 
- Vérifiez que la compilation soit marquée comme « [SUCCESS] »

```
Build Build Output x
PlatformIO Build: finished 22 sec, 771 ms
Linking .pio\build\esp12e\firmware.elf
Retrieving maximum program size .pio\build\esp12e\firmware.elf
Checking size .pio\build\esp12e\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [=== ] 34.3% (used 28108 bytes from 81920 bytes)
Flash: [=== ] 25.0% (used 261515 bytes from 1044464 bytes)
Building .pio\build\esp12e\firmware.bin
Creating BIN file ".pio\build\esp12e\firmware.bin" using "C:\Users\...\.platformio\
[SUCCESS] Took 21.41 seconds
```

- Téléversez le code compilé dans votre ESP8266 en cliquant sur 
- Contemplez la LED native de votre ESP8266 qui clignote toutes les 500ms



Bravo, vous venez de programmer votre premier IoT et l'ensemble de votre environnement de développement et opérationnel. Nous allons pouvoir passer aux choses sérieuses.